Reproduce-then-Extend — Ames House Prices (Real Estate / Hedonic Economics) (Day 1)

Intro to Machine Learning · Bruce A. Desmarais

July 04, 2026

Published study. De Cock, D. (2011). "Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project." *Journal of Statistics Education* 19(3). The analysis is an **OLS hedonic price regression** of sale price on property features (n = 2930; data at the JSE site). This example illustrates **regularization and variable selection** in a wide model.

1 Background

De Cock introduces the **Ames housing dataset** as a modern, richer alternative to the classic Boston housing data for teaching regression. The intended use of the data is a **hedonic price regression** – an OLS model that decomposes a home's **sale price** into the contributions of its individual characteristics (size, quality, age, amenities). The **primary conclusion** is methodological: a well-specified hedonic OLS explains the large majority of the variation in sale price (R^2 around 0.9), and the data support a full end-of-semester regression project from specification through diagnostics.

2 Data and codebook

Unit of analysis: a single-family residential **home sale**; n = 2,930. The printed hedonic regression uses ten interpretable features; the regularization section uses De Cock's full ~80-feature set.

Variable	Definition
`SalePrice`	sale price in USD **(outcome)**
`Overall.Qual`	overall material-and-finish quality (1-10)
`Gr.Liv.Area`	above-grade living area (sq ft)
`Total.Bsmt.SF`	total basement area (sq ft)
`Garage.Cars`	garage capacity (number of cars)
`Year.Built`	year of original construction
`Year.Remod.Add`	year of the last remodel/addition
`Full.Bath`	full bathrooms above grade
`TotRms.AbvGrd`	total rooms above grade (excludes bathrooms)
`Lot.Area`	lot size (sq ft)
`Fireplaces`	number of fireplaces

3 Descriptive results

```
d <- read.delim("data/AmesHousing.txt",
                stringsAsFactors = FALSE, check.names = TRUE)
feats <- c("Overall.Qual", "Gr.Liv.Area", "Total.Bsmt.SF", "Garage.Cars", "Year.Built",
```

```
          "Year.Remod.Add", "Full.Bath", "TotRms.AbvGrd", "Lot.Area", "Fireplaces")
for (j in feats) d[[j]][is.na(d[[j]])] <- median(d[[j]], na.rm = TRUE)   # a few missing
↪  values
d <- d[complete.cases(d[, c("SalePrice", feats)]), ]

round(t(sapply(d[c("SalePrice", feats)],
               function(x) c(mean = mean(x), sd = sd(x), min = min(x), max = max(x)))),
               ↪  1)
```

```
##                    mean      sd   min    max
## SalePrice      180796.1 79886.7 12789 755000
## Overall.Qual        6.1     1.4     1     10
## Gr.Liv.Area      1499.7   505.5   334   5642
## Total.Bsmt.SF    1051.6   440.5     0   6110
## Garage.Cars         1.8     0.8     0      5
## Year.Built       1971.4    30.2  1872   2010
## Year.Remod.Add   1984.3    20.9  1950   2010
## Full.Bath           1.6     0.6     0      4
## TotRms.AbvGrd       6.4     1.6     2     15
## Lot.Area        10147.9  7880.0  1300 215245
## Fireplaces          0.6     0.6     0      4
```

```
clean <- function(v) gsub("[._]+", " ", v)
par(mfrow = c(1, 2))
par(mar = c(4, 4, 2.5, 1))
hist(d$SalePrice, breaks = 30, col = "#a6cee3", border = "white", main = "Outcome:
↪  SalePrice",
     xlab = "sale price (USD)")
cors <- sort(sapply(d[feats], function(x) cor(x, d$SalePrice)))
par(mar = c(4, 7, 2.5, 1))
barplot(cors, horiz = TRUE, las = 1, names.arg = clean(names(cors)), cex.names = 0.8,
        col = "#1f78b4", main = "Correlation with price", xlab = "correlation")
```

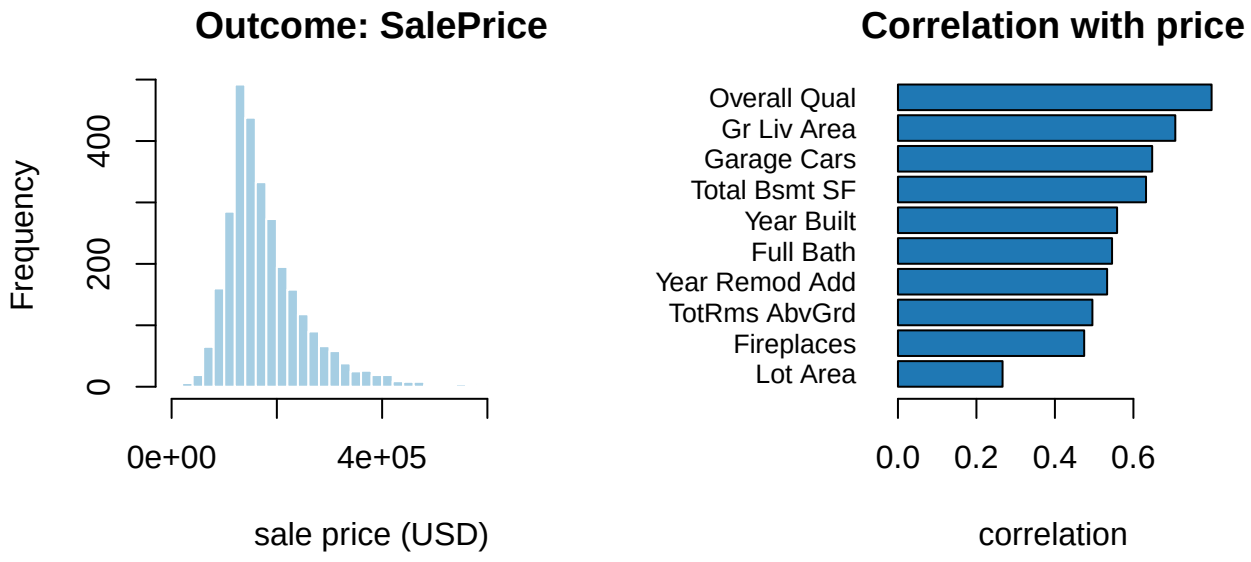

Figure 1: Sale-price distribution (left) and each feature's correlation with price (right).

4 Reproduce the published regression

A standard hedonic OLS on interpretable features (De Cock's regression-project design).

```
hedonic <- lm(SalePrice ~ ., data = d[, c("SalePrice", feats)])
summary(hedonic)
```

```
##
## Call:
## lm(formula = SalePrice ~ ., data = d[, c("SalePrice", feats)])
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -530526  -19202   -2071   15112  279889
##
## Coefficients:
##                   Estimate Std. Error t value Pr(>|t|)
## (Intercept)    -1.289e+06  8.531e+04 -15.112  < 2e-16 ***
## Overall.Qual    1.925e+04  7.801e+02  24.676  < 2e-16 ***
## Gr.Liv.Area     5.307e+01  2.850e+00  18.622  < 2e-16 ***
## Total.Bsmt.SF   2.925e+01  1.908e+00  15.333  < 2e-16 ***
## Garage.Cars     1.202e+04  1.188e+03  10.116  < 2e-16 ***
## Year.Built      2.783e+02  3.322e+01   8.378  < 2e-16 ***
## Year.Remod.Add  3.435e+02  4.306e+01   7.977 2.13e-15 ***
## Full.Bath      -7.246e+03  1.742e+03  -4.161 3.27e-05 ***
## TotRms.AbvGrd  -1.531e+03  7.324e+02  -2.091   0.0366 *
## Lot.Area        6.725e-01  9.078e-02   7.408 1.67e-13 ***
## Fireplaces      8.800e+03  1.202e+03   7.319 3.21e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 35690 on 2919 degrees of freedom
## Multiple R-squared:  0.8011, Adjusted R-squared:  0.8004
## F-statistic:  1176 on 10 and 2919 DF,  p-value: < 2.2e-16
```

Confirmation against the published study. In-sample $R^2 = 0.801$, with **overall quality** and **above-grade living area** the dominant positive drivers of price. (Adding the full ~80-feature set with neighborhood indicators raises the OLS to $R^2 \approx 0.92$, matching De Cock's larger model.)

5 Regularization & variable selection

Regularization's natural home is a **wide** model. We expand to the full feature set — every numeric and one-hot-encoded categorical predictor (~250 columns) — where ordinary least squares over-fits.

```
# build the full design matrix: numeric features as-is, categoricals one-hot encoded
full <- d[, !names(d) %in% c("Order", "PID")]
y    <- full$SalePrice; full$SalePrice <- NULL
for (j in names(full)) {
  if (is.character(full[[j]])) { full[[j]][is.na(full[[j]])] <- "NA"; full[[j]] <-
  ↪  factor(full[[j]]) }
  else full[[j]][is.na(full[[j]])] <- median(full[[j]], na.rm = TRUE)
}
X <- model.matrix(~ . - 1, data = full)
X <- X[, apply(X, 2, function(z) length(unique(z)) > 1)]   # drop constant columns
cat(sprintf("Full design matrix: %d homes x %d features\n", nrow(X), ncol(X)))
```

```
## Full design matrix: 2930 homes x 286 features
```

```
set.seed(2026)
lasso <- cv.glmnet(X, y, alpha = 1)     # L1
ridge <- cv.glmnet(X, y, alpha = 0)     # L2
enet  <- cv.glmnet(X, y, alpha = 0.5)   # L1 + L2
n_keep <- sum(coef(lasso, s = "lambda.1se")[-1] != 0)   # nonzero features at the 1-SE
↪  penalty
cat(sprintf("At lambda.1se the lasso keeps %d of %d features (the rest are set to 0);
↪  ridge keeps all but shrinks them.\n",
            n_keep, ncol(X)))
```

```
## At lambda.1se the lasso keeps 39 of 286 features (the rest are set to 0); ridge keeps
↪  all but shrinks them.
```

Which fits best out of sample? We hold out a random 30% test set, fit each model on the training 70% (penalty tuned by CV inside the training data), and compare held-out **RMSE** (dollars) on the untouched test set; we **repeat the split 10 times and average**. Penalized models are scored at `lambda.min`, the loss-minimizing penalty (the right choice for *prediction*; the sparser `lambda.1se` used for the coefficient display trades a little accuracy for parsimony). On ~250 features unpenalized OLS over-fits badly.

```
rmse   <- function(p, yt) sqrt(mean((yt - p)^2))
REPS   <- 10
alphas <- c(Lasso = 1, Ridge = 0, ElasticNet = 0.5)
Xdf <- as.data.frame(X)
M <- matrix(NA, REPS, 4, dimnames = list(NULL, c("OLS", names(alphas))))
for (r in 1:REPS) {
  set.seed(2025 + r)
  te <- sample(nrow(X), round(0.30 * nrow(X))); tr <- setdiff(seq_len(nrow(X)), te)
  ols <- lm(y[tr] ~ ., data = Xdf[tr, ])                                    # baseline
↪  OLS
  M[r, "OLS"] <- rmse(predict(ols, Xdf[te, ]), y[te])
  for (a in names(alphas)) {
    cv <- cv.glmnet(X[tr, ], y[tr], alpha = alphas[a])
    M[r, a] <- rmse(as.numeric(predict(cv, X[te, ], s = "lambda.min")), y[te])
  }
}
round(colMeans(M))                                                  # mean held-out RMSE ($)
↪  over 10 splits
```

```
##        OLS      Lasso      Ridge ElasticNet
##      35257      30225      30188      30437
```

```
best <- names(which.min(colMeans(M)[names(alphas)]))
cat(sprintf("\nBest penalization (10 splits): %s (mean test RMSE = $%s vs OLS $%s).\n",
            best, format(round(min(colMeans(M)[names(alphas)])), big.mark = ","),
            format(round(colMeans(M)["OLS"]), big.mark = ",")))
```

```
##
## Best penalization (10 splits): Ridge (mean test RMSE = $30,188 vs OLS $35,257).
```

On ~250 features OLS over-fits, but lasso/ridge/elastic-net keep only a few dozen effective predictors and **cut held-out RMSE well below OLS** — regularization buying both parsimony and accuracy. This is the setting (wide, collinear design) where a penalty pays off in prediction, in contrast to compact, curated models with large samples where a penalty mainly buys parsimony rather than accuracy.

To see the shrinkage concretely, we refit the three penalties on the **ten interpretable features** and place the penalized coefficients next to the original OLS (dollars per unit):

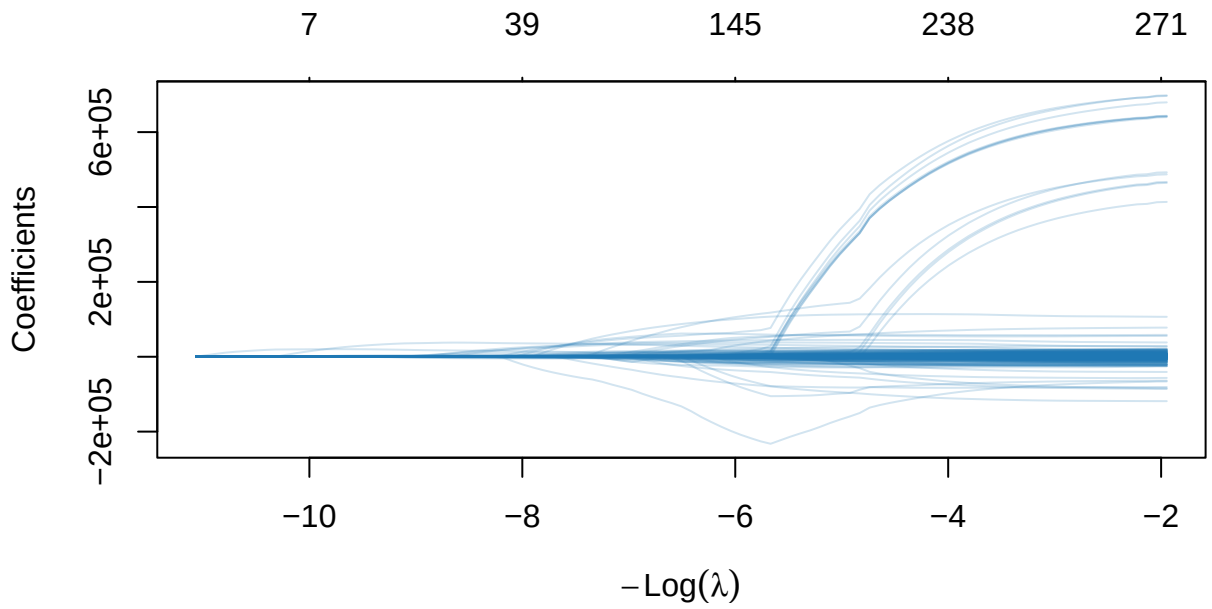

Figure 2: Lasso coefficient paths vs the penalty (log lambda) on the full Ames feature set; the dashed line marks the CV-optimal penalty.

```
Xi <- as.matrix(d[, feats])
li <- cv.glmnet(Xi, y, alpha = 1); ri <- cv.glmnet(Xi, y, alpha = 0); ei <- cv.glmnet(Xi,
↪  y, alpha = 0.5)
coefs_at_1se <- function(cv) as.numeric(coef(cv, s = "lambda.1se"))[-1]   # 1-SE penalty
data.frame(
  OLS        = round(coef(hedonic)[-1]),
  Lasso      = round(coefs_at_1se(li)),
  Ridge      = round(coefs_at_1se(ri)),
  ElasticNet = round(coefs_at_1se(ei)),
  row.names = feats)
```

```
##                  OLS Lasso Ridge ElasticNet
## Overall.Qual   19249 20510 11515      17862
## Gr.Liv.Area       53    41    25         39
## Total.Bsmt.SF     29    25    26         26
## Garage.Cars    12020 10096 12457      11254
## Year.Built       278   127   259        166
## Year.Remod.Add   343   104   363        195
## Full.Bath      -7246     0  7022          0
## TotRms.AbvGrd  -1531     0  3293          0
## Lot.Area           1     0     1          0
## Fireplaces      8800  2825 11503       5603
```

Each penalty pulls the OLS dollar-coefficients toward zero (ridge most uniformly); at the 1-SE penalty the lasso drops the weakest of the ten to exactly zero, and on the full ~250-column model above it zeroes the large majority of columns.

6 Best-subset selection with ABESS

Lasso reaches a sparse model *indirectly* through shrinkage. **Best-subset selection** instead searches directly for the subset of features that fits best, and — via the **adaptive best-subset (ABESS)** algorithm — does

so in polynomial time even on this wide design, choosing the subset size automatically.

```
library(abess)
ab <- abess(X, y)                                    # X = full ~250-feature design matrix
cat(sprintf("ABESS selects a best subset of %d of %d features. Top few by
↪  |coefficient|:\n",
            ab$best.size, ncol(X)))
```

```
## ABESS selects a best subset of 92 of 286 features. Top few by |coefficient|:
```

```
cf  <- as.numeric(coef(ab, support.size = ab$best.size))[-1]
top <- order(abs(cf), decreasing = TRUE)[1:8]
data.frame(feature = colnames(X)[top], coef = round(cf[top]))
```

```
##            feature   coef
## 1 Roof.MatlWdShngl 597446
## 2 Roof.MatlMembran 584537
## 3   Roof.MatlMetal 568418
## 4    Roof.MatlRoll 542421
## 5 Roof.MatlCompShg 542080
## 6 Roof.MatlWdShake 538803
## 7 Roof.MatlTar&Grv 538024
## 8 Misc.FeatureGar2 161100
```

Best subset and the L_1 path agree on the price drivers — overall quality, living area, and the strongest neighborhood and condition indicators — reached by direct search rather than shrinkage.

7 Takeaway

This example illustrates **regularization and variable selection** in the setting it was built for: a wide hedonic model where unpenalized OLS over-fits. Lasso/elastic-net (`glmnet`) and best-subset selection (`abess`) each pick a compact, accurate model from ~250 candidate features — the principled way to handle many correlated predictors.

8 Recommended exercises

1. Compare lasso, ridge, and elastic net out-of-sample RMSE on identical folds; which wins?
2. Add interaction or polynomial terms for a couple of strong predictors and see whether penalized RMSE improves.
3. Plot the lasso coefficient path and note the order in which variables enter the model.